

1

Logger

LOGGER

You are working on a project that requires the use of a logging framework. Write a Java program to create an interface called "Logger" with methods for logging different types of messages, such as "debug", "info", "warning", and "error". Implement the

```

1 interface Logger {
2     void debug(String message);
3     void info(String message);
4     void warning(String message);
5     void error(String message);
6 }
7
8 public class Log4jLogger implements Logger {
9
10    public void debug(String message) {
11        System.out.println("[LOG4J DEBUG] " + message);
12    }
13
14    public void info(String message) {
15        System.out.println("[LOG4J INFO] " + message);
16    }
17
18    public void warning(String message) {
19        System.out.println("[LOG4J WARNING] " + message);
20    }
21

```

OUTPUT

```

[LOG4J DEBUG] Debugging application.
[LOG4J INFO] Application started successfully.
[LOG4J WARNING] Low disk space.
[LOG4J ERROR] Database connection failed.

```

Your INPUT goes here!

2.

Character

CHARACTER

You are developing a game that involves different types of characters, such as heroes, villains, and monsters. Write a Java program to create an interface called "Character" with methods for moving, attacking, and taking damage. Implement the interface in classes

```

1 interface Character {
2     void move();
3     void attack();
4     void takeDamage();
5 }
6
7 public class Hero implements Character {
8
9     public void move() {
10        System.out.println("Hero moves.");
11    }
12
13    public void attack() {
14        System.out.println("Hero attacks.");
15    }
16
17    public void takeDamage() {
18        System.out.println("Hero takes damage.");
19    }
20
21

```

OUTPUT

```

Hero moves.
Hero attacks.
Hero takes damage.

```

Your INPUT goes here!

3.

Sort

SORT

Write a Java program to create a list of integers and sort them in ascending order using the Collections framework.

```

1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner (System.in);
5         ArrayList<Integer>list=new ArrayList<Integer>();
6         System.out.println("enter 5 integers:");
7         for(int i=0;i<5;i++){
8             list.add(sc.nextInt());
9         }
10        Collections.sort(list);
11        System.out.println("sorted list:"+list);
12    }
13 }

```

50

20

40

10

30

OUTPUT

```

enter 5 integers:
sorted list:[10, 20, 30, 40, 50]

```

4.

QUESTIONS 8 / 8 completed

HashSet

HASHSET

Write a Java program to create a HashSet of strings and remove a specific element from it.

PROGRAM EDITOR

Java

Run

Save

```

1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         HashSet<String> set = new HashSet<>();
7
8         set.add("Apple");
9         set.add("Banana");
10        set.add("Mango");
11        set.add("Orange");
12        set.add("Grapes");
13
14        System.out.println("Before Removal: " + set);
15
16        set.remove("Mango");
17
18        System.out.println("After Removal: " + set);
19    }
20 }

```

OUTPUT

INPUT

Apple

Banana

Mango

Orange

Grapes

Mango

5.

TreeMap

TREEMAP

Write a Java program to create a TreeMap of strings and integers and iterate over its entries.

```

1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7         TreeMap<String, Integer> map = new TreeMap<>();
8
9         System.out.println("Enter 3 names and marks:");
10
11         for (int i = 0; i < 3; i++) {
12             String name = sc.next();
13             int marks = sc.nextInt();
14             map.put(name, marks);
15         }
16
17         System.out.println("TreeMap Entries:");
18
19         for (Map.Entry<String, Integer> entry : map.entrySet()) {
20             System.out.println(entry.getKey() + " : " + entry.getValue());
21         }
22     }
23 }

```

OUTPUT

```

Enter 3 names and marks:
TreeMap Entries:
arun : 85
bala : 95

```

ram 90

arun 85

bala 95

6.

PriorityQueue

PRIORITYQUEUE

Write a Java program to create a PriorityQueue of integers and add elements to it.

```

1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc = new Scanner(System.in);
5         PriorityQueue<Integer> pq = new PriorityQueue<>();
6         System.out.println("enter 5 integers:");
7         for(int i=0; i<5; i++){
8             pq.offer(sc.nextInt());
9         }
10        System.out.println("Priority Queue :"+pq);
11    }
12 }

```

OUTPUT

```

enter 5 integers:
Priority Queue :[10, 20, 40, 50, 30]

```

50

20

40

10

30

7.

LinkedHashMap

LINKEDHASHMAP

Write a Java program to create a LinkedHashMap of strings and integers and retrieve the keys in the order they were inserted.

```

1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
7
8         map.put("Ram", 90);
9         map.put("Arun", 85);
10        map.put("Bala", 95);
11
12        System.out.println("Keys in insertion order:");
13
14        for (String key : map.keySet()) {
15            System.out.println(key);
16        }
17    }
18 }

```

OUTPUT

```

Keys in insertion order:
Ram
Arun
Bala

```

Ram 90

Arun 85

Bala 95

8.

Max-Min

MAX-MIN

Write a Java program to create a Set of integers and perform different operations on it, such as finding the maximum and minimum values.

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         HashSet<Integer> set = new HashSet<>();
7
8         set.add(50);
9         set.add(20);
10        set.add(40);
11        set.add(10);
12        set.add(30);
13
14        System.out.println("Set: " + set);
15        System.out.println("Maximum Value: " + Collections.max(set));
16        System.out.println("Minimum Value: " + Collections.min(set));
17    }
18 }
```

OUTPUT

```
Set: [50, 20, 40, 10, 30]
Maximum Value: 50
Minimum Value: 10
```

Your INPUT goes here!